

Multitenant Architecture

Oracle Multitenant is a new architecture in Oracle database 12c to support cloud infrastructure and consolidation strategies in the data centers.

Multi-Tenant – Multi-tenancy means that **a single instance of the software and its supporting infrastructure serves multiple customers.**

The **multitenant architecture** enables an Oracle database to function as a multitenant container database (CDB).

A **CDB** includes zero, one, or many customer-created pluggable databases (PDBs). A **PDB** is a portable collection of schemas, schema objects, and nonschema objects that appears to an Oracle Net client as a **non-CDB**. All Oracle databases before Oracle Database 12c were non-CDBs.

The CDB: The CDB represents the database as a whole. It contains multiple PDBs. It contains the metadata off all the PDBs.

PDB: A PDB is designed to provide what is essentially a unique and isolated database environment within a CDB.

Like a normal database, a PDB contains physical datafiles that store the data contained within that PDB. The datafiles also contain a local copy of the data dictionary, which contains the metadata associated with that PDB. The data in the PDB is isolated to the PDB itself, and is not shared across other PDBs.

All PDBs are owned by the parent CDB SYS user. A given CDB can have up to **252 PDBs** plugged into it at any one time. Each PDB has, at a minimum, its own SYSTEM, SYSAUX, and USERS tablespaces. You can create additional tablespaces in a PDB just as in any other database.

Each pdb has an associated service name assigned to it. So if you want to connect to PDB directly use the service name(tns name) to connect like

Root container: Each CDB has exactly one root container. This container provides a location for schemas, schema objects, and non-schema objects that belong to the CDB as a whole. System metadata required for the database to manage the individual PDBs are stored in the root container. The root container is named CDB\$ROOT.

Seed PDB: Each CDB has an Oracle supplied PDB called the seed. This PDB is called PDB\$SEED and it's used to create new PDBs within the CDB. You cannot add or modify objects in PDB\$SEED.

A PDB don't have background processes, redo logs and control file. It uses that of CDB.

CDB can be opted for archivelog mode. This mode itself is done at the level of the CDB. You cannot opt to disable ARCHIVELOG mode for individual PDBs.

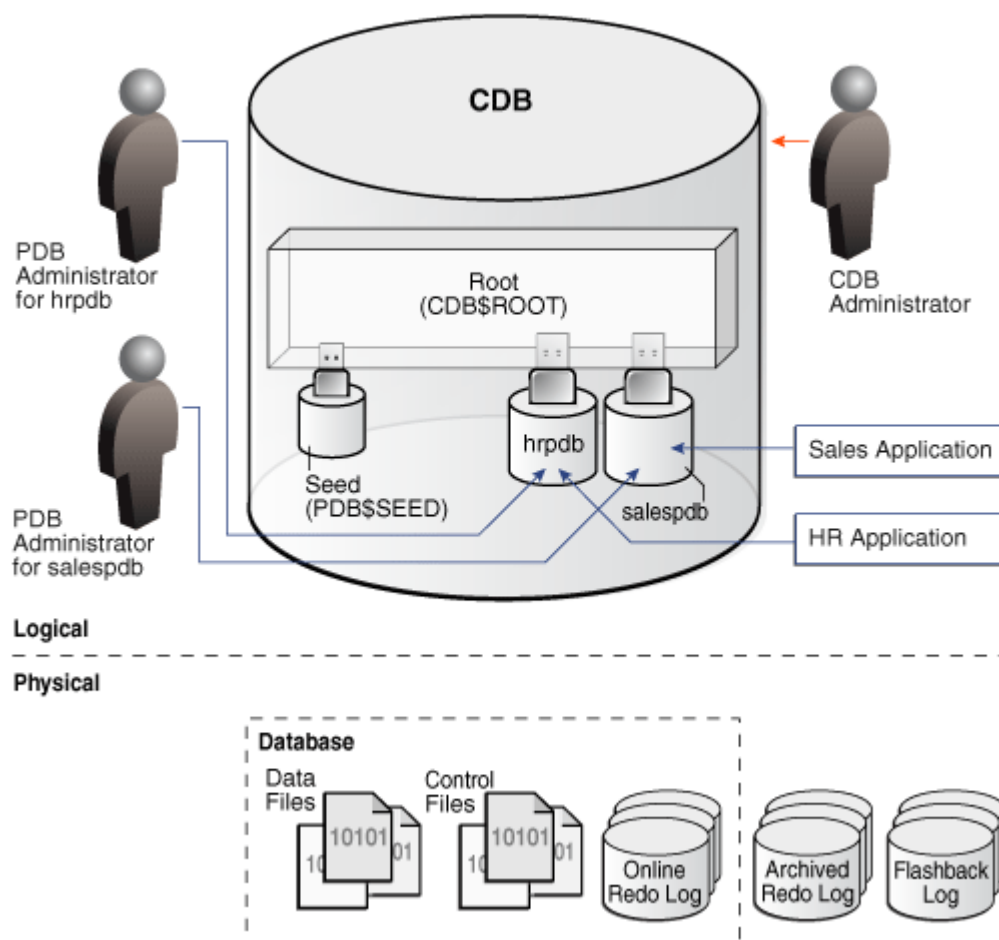
How to check whether the database is multitenant or not:

```
SQL> select NAME, DECODE(CDB, 'YES', 'Multitenant Option enabled', 'Regular 12c Database: ') "Multitenant Option ?" , OPEN_MODE, CON_ID from V$DATABASE;
```

```
NAME Multitenant Option ? OPEN_MODE CON_ID
```

```
DBATEST Multitenant Option enabled READ WRITE 0
```

Figure 17-1 CDB with Two PDBs



Benefits of the Multitenant Architecture

The major benefits of Oracle Multitenant option is easy adoption, manageability, database isolation and security, and resource prioritization for each database.

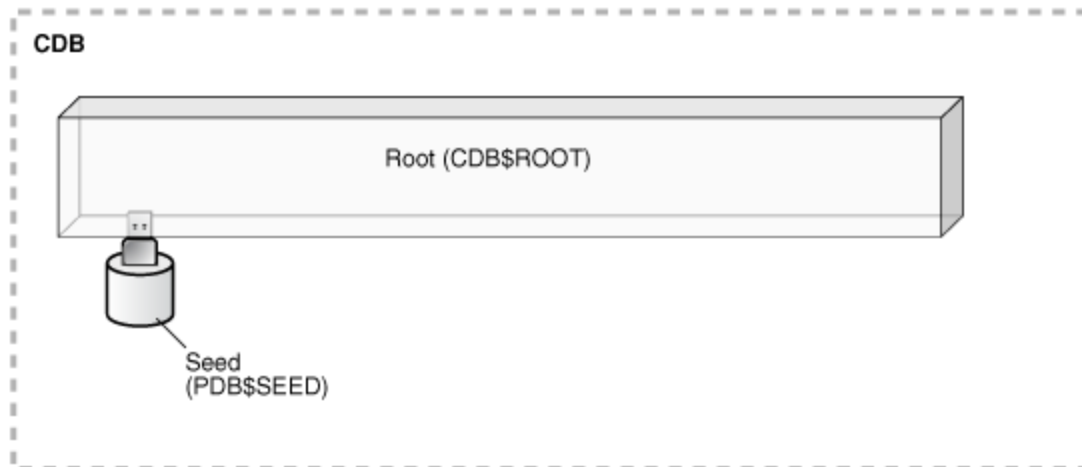
- Cost reduction
 - By consolidating hardware and database infrastructure to a single set of background processes, and efficiently sharing computational and memory resources, you reduce costs for hardware and maintenance. For example, 100 PDBs on a single server share one database instance.
- Easier and more rapid movement of data and code
 - By design, you can plug a PDB into a CDB, unplug the PDB from the CDB, and then plug this PDB into a different CDB.
- Easier management and monitoring of the physical database
 - The CDB administrator can manage the environment as an aggregate by executing a single operation, such as patching or performing an RMAN backup, for all hosted tenants and the CDB root. Backup strategies and disaster recovery are simplified.
- Ease of performance tuning
 - It is easier to collect performance metrics for a single database than for multiple databases. It is easier to size one SGA than 100 SGAs.
- Fewer database patches and upgrades
 - It is easier to apply a patch to one database than to 100 databases, and to upgrade one database than to upgrade 100 databases.
- Easier migration of data and code
 - For example, instead of upgrading a CDB from one database release to another, you can unplug a PDB from the existing CDB, and then plug it into a newly created CDB from a higher release.
- Easier testing of applications
 - You can develop an application on a test PDB and, when it is ready for deployment, plug this PDB into the production CDB.

Creation of a CDB

You must define a database as a CDB at creation, and then create PDBs and application containers within this CDB.

The `CREATE DATABASE ... ENABLE PLUGGABLE DATABASE SQL` statement creates a new CDB. If you do not specify the `ENABLE PLUGGABLE DATABASE` clause, then the newly created database is a non-CDB and can never contain PDBs.

Along with the root container (`CDB$ROOT`), Oracle Database automatically creates a seed PDB (`PDB$SEED`). The following graphic shows a newly created CDB:



Description of "Figure 17-4 CDB with Seed PDB"

```
SQL> SELECT NAME, CDB, CON_ID FROM V$DATABASE;
```

NAME	CDB	CON_ID
-----	----	-----
CDB1	YES	0

Using DBCA to Create a CDB

Oracle strongly recommends using the Database Configuration Assistant (DBCA) to create a CDB, because it is a more automated approach, and your CDB is ready to use when DBCA completes. DBCA enables you to specify the number of PDBs in the CDB when it is created.

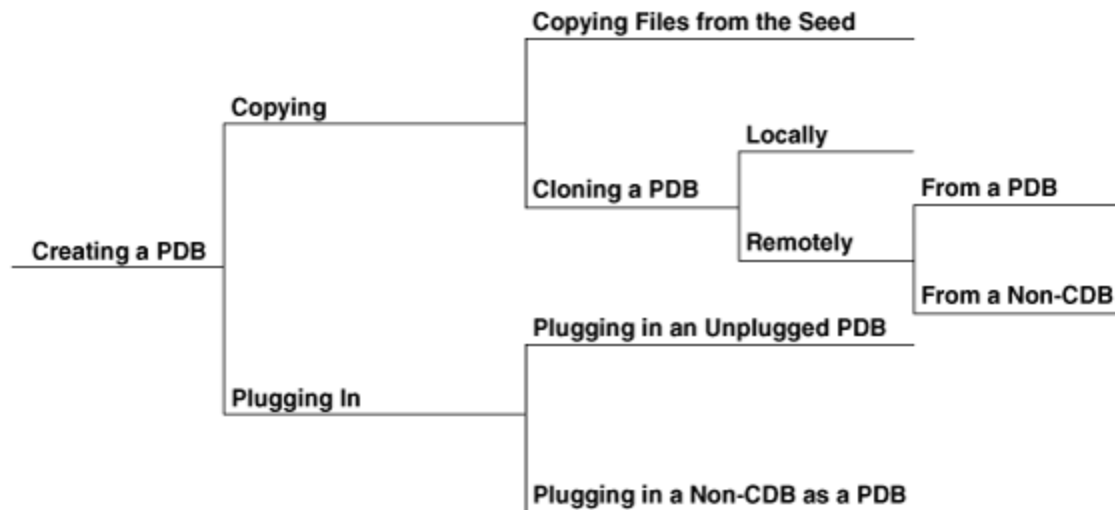
DBCA can be launched by the Oracle Universal Installer (OUI), depending upon the type of install that you select. You can also launch DBCA as a standalone tool at any time after Oracle Database installation.

After a CDB is created, you can use DBCA to plug PDBs into it and unplug PDBs from it.

Creation of a PDB

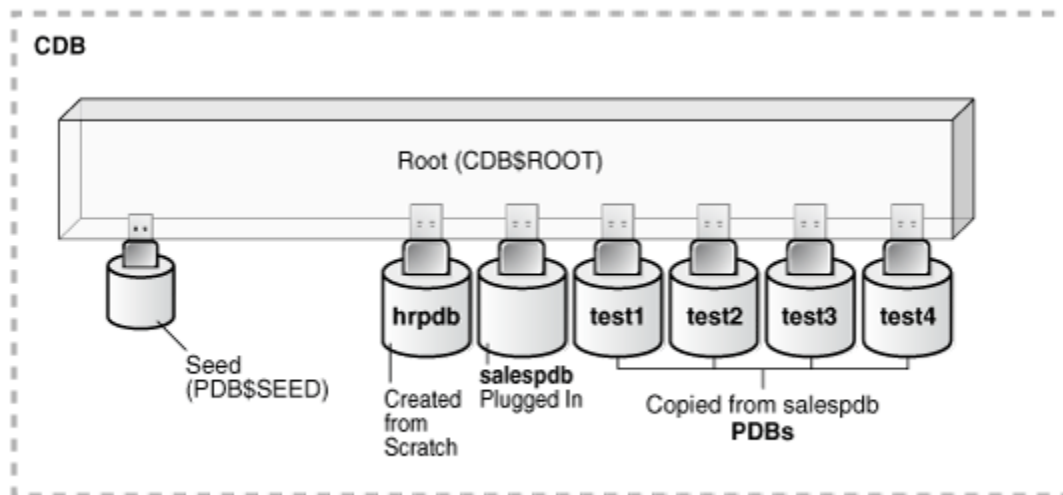
The `CREATE PLUGGABLE DATABASE SQL` statement creates a PDB. This PDB automatically includes a full data dictionary including metadata and internal links to system-supplied objects in the root. You can only create a PDB in a CDB and not within another PDB.

The following figure depicts the options for creating a PDB.



Description of "Figure 17-5 Options for Creating a PDB"

The following figure shows a CDB that contains six PDBs. hrpdb is a newly created PDB. salespdb was a pre-existing PDB that was unplugged from a different CDB and plugged into this CDB. The remaining four PDBs, each of whose names contains the prefix test, were copied from salespdb.

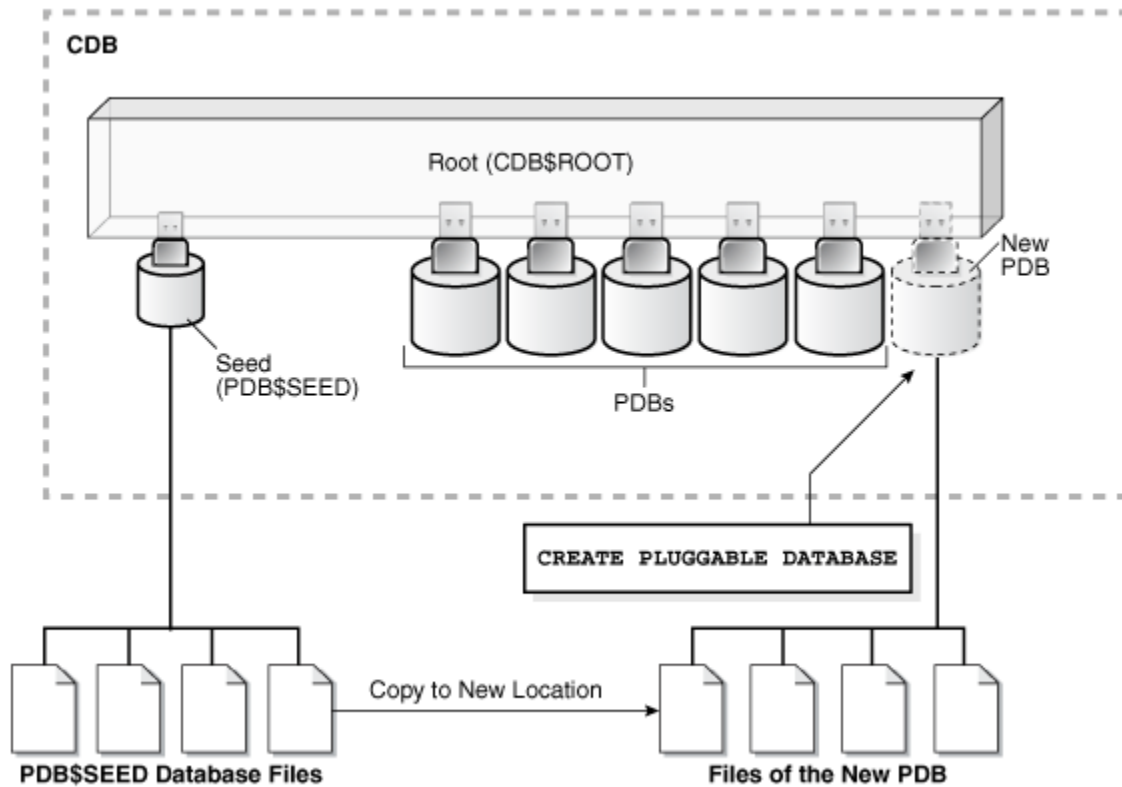


Description of "Figure 17-6 CDB with Six PDBs"

Creation of a PDB from Seed

You can use the `CREATE PLUGGABLE DATABASE` statement to create a PDB by copying the files from `PDB$SEED`, which is a template for creating PDBs.

The following figure illustrates creation from the seed.



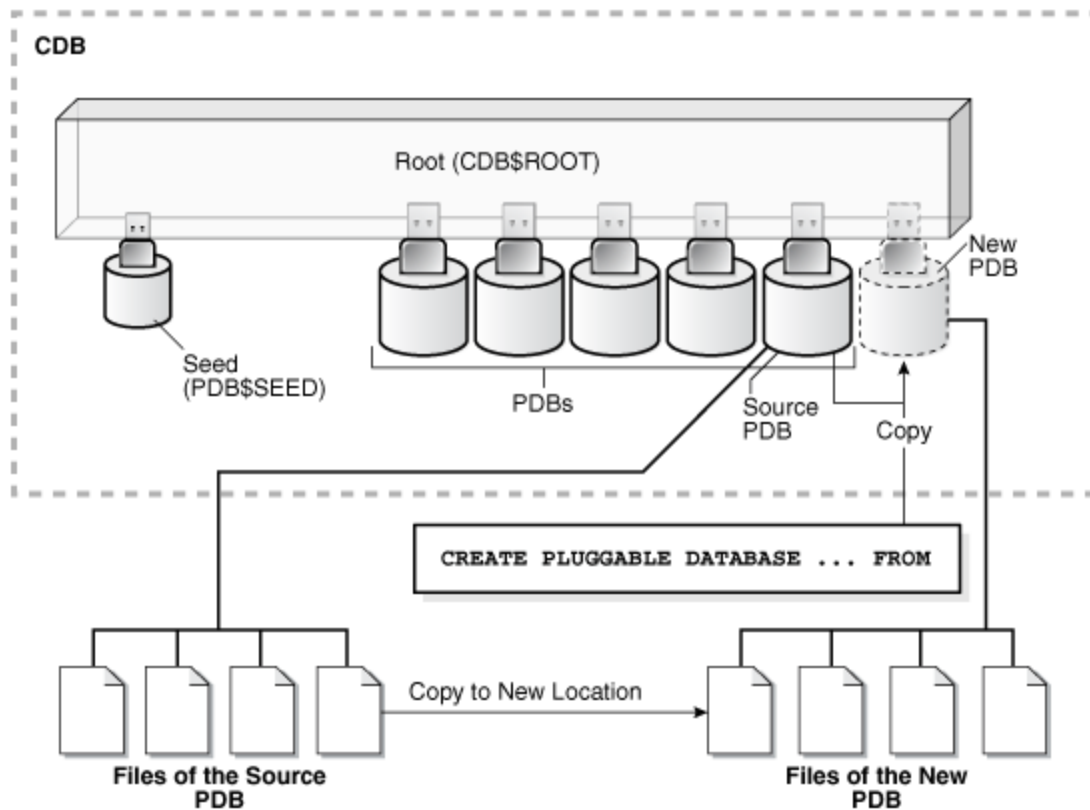
Description of "Figure 17-7 Creation of a PDB from the Seed PDB"

```
CREATE PLUGGABLE DATABASE hrpdb
```

```
ADMIN USER dba1 IDENTIFIED BY password
```

Creation of a PDB by Cloning a PDB or a Non-CDB

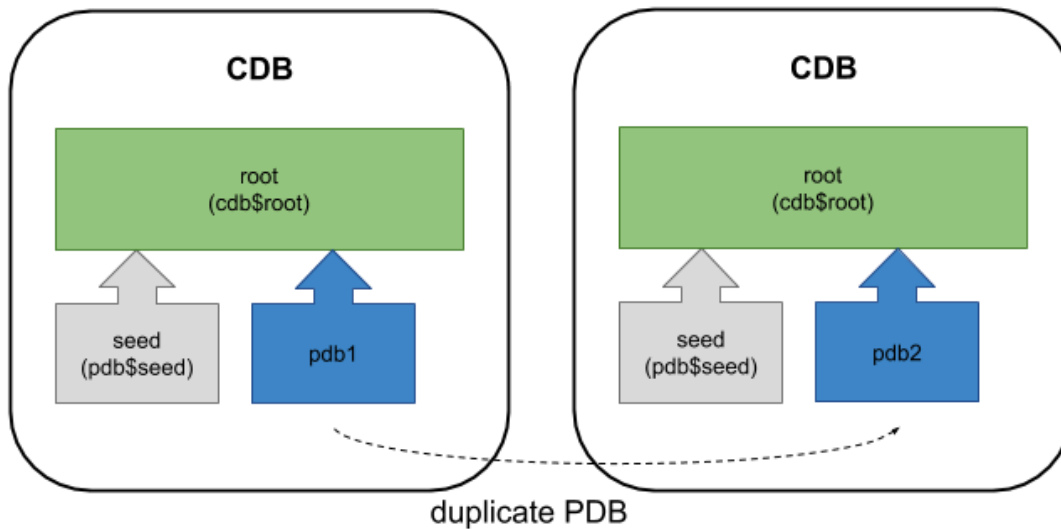
You can use the CREATE PLUGGABLE DATABASE statement to clone a source PDB or non-CDB and plug the clone into the CDB.



Description of "Figure 17-8 Cloning a PDB from a PDB in the Same CDB"

Duplicating Databases

Database duplication is the use of the `Duplicate` command to copy all or a subset of the data in a source database. The duplicate database (the copied database) functions entirely independently from the source database (the database being copied).



Purpose of Database Duplication

A duplicate database is useful for a variety of purposes, most of which involve testing. You can perform the following tasks in a duplicate database:

- Test backup and recovery procedures
- Test an upgrade to a new release of Oracle Database
- Test the effect of applications on database performance
- Create a standby database
- Generate reports

Types of Database Duplication

RMAN enables you to perform two main types of database duplication.

They include the following:

- **Backup-based duplication**

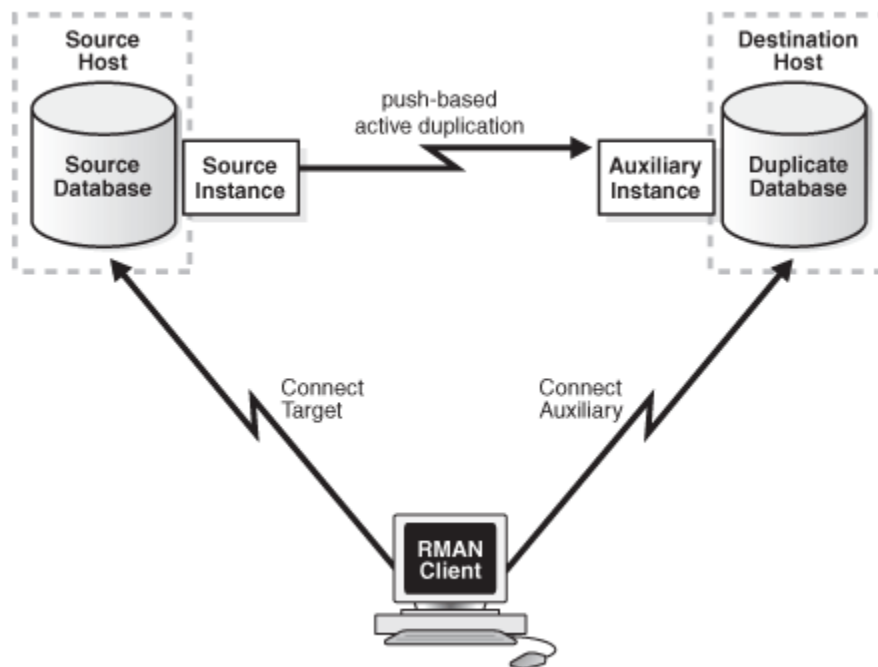
The duplicate database is created by using preexisting RMAN backups or copies of the source database. You can use different techniques to duplicate a database by using backup-based duplication.

- **Active database duplication**

The duplicate database is created by copying the live source database over the network to the auxiliary instance. The duplication can be performed by using backup sets or image copies.

You can use any type of duplication to duplicate a database either to the local host or to a remote host.

The DUPLICATE command can create a fully functional copy of your database or create a **physical standby database**, which serves a very different purpose. A standby database is a copy of the primary database that you update continually with archived redo log files from the primary database. If the primary database is inaccessible, then you can fail over to the standby database, which becomes the new primary database.



Description of "Figure 25-2 Active Database Duplication"

When active database duplication is performed using image copies, after RMAN establishes a connection with the source database, the source database transfers the required database files to the auxiliary database. Using image copies may require additional resources on the source database. This method is referred to as the **push-based method** of active database duplication.

When RMAN performs active database duplication using backup sets, a connection is established with the source database and the auxiliary database. The auxiliary database then connects to the source database through Oracle Net Services and retrieves the required database files from the source database. This method of active database duplication is also referred to as the **pull-based method**.

A duplicate database can include the same contents as the source database or only a subset of the tablespaces in the source database. For example, you can use the TABLESPACE option of the DUPLICATE command to duplicate only specified tablespaces, or the SKIP READONLY option to exclude read-only tablespaces from the duplicate database.

Basic Steps of Database Duplication

1. Prepare for database duplication.
2. Start RMAN and connect to the necessary database instances.
3. For active database duplication, decide if you want to encrypt or compress the backup sets that are used to transfer files from the source database to the duplicate database. Additionally, decide if you want to create backup sets on the source database in parallel by using multisection backups.
4. Place the source database in the proper state (if necessary).
5. Configure RMAN channels (if necessary).
6. Perform the duplication.

```
DUPLICATE TARGET DATABASE TO dupdb FROM ACTIVE DATABASE  
  
PASSWORD FILE  
  
SPFILE  
  
NOFILENAMECHECK;
```

This example illustrates how to perform active duplication. RMAN uses the pull method (using backup sets) by default. `DUPLICATE` requires the `NOFILENAMECHECK` option because the source database files have the same names as the duplicate database files.

The `PASSWORD FILE` option specifies that RMAN copies the password file to the destination host.

RMAN automatically copies the server parameter file to the destination host, restarts the auxiliary instance with the server parameter file, copies all necessary database files and archived redo logs over the network to the destination host, and recovers the database. Finally, RMAN opens the database with the `RESETLOGS` option to create the online redo log. `dupdb` is the `DB_NAME` of the duplicate database

- To duplicate the PDB `pdb1` to the CDB `cdb1`, use the following command:
- `DUPLICATE DATABASE TO cdb1 PLUGGABLE DATABASE pdb1;`

Managing Security for a Multitenant Environment

You can manage common and local users and roles for a multitenant environment by using SQL*Plus and Oracle Enterprise Manager.

- Common User : The user is present in all containers (root and all PDBs).
- Local User : The user is only present in a specific PDB. The same username can be present in multiple PDBs, but they are unrelated.

Likewise, there are two types of roles.

- Common Role : The role is present in all containers (root and all PDBs).
- Local Role : The role is only present in a specific PDB. The same role name can be used in multiple PDBs, but they are unrelated.

Create Common Users

When creating a common user the following requirements must all be met.

- You must be connected to a common user with the CREATE USER privilege.
- The current container must be the root container.
- The username for the common user must be prefixed with "C##" or "c##" and contain only ASCII or EBCDIC characters.
- The username must be unique across all containers.
- The DEFAULT TABLESPACE, TEMPORARY
TABLESPACE, QUOTA and PROFILE must all reference objects that exist in all containers.
- You can either specify the CONTAINER=ALL clause, or omit it, as this is the default setting when the current container is the root.

- CONN / AS SYSDBA
-
- -- Create the common user using the CONTAINER clause.
- CREATE USER c##test_user1 IDENTIFIED BY password1 CONTAINER=ALL;
- GRANT CREATE SESSION TO c##test_user1 CONTAINER=ALL;
-
- -- Create the common user using the default CONTAINER setting.
- CREATE USER c##test_user2 IDENTIFIED BY password1;
- GRANT CREATE SESSION TO c##test_user2;

Create Local Users

When creating a local user the following requirements must all be met.

- You must be connected to a user with the CREATE USER privilege.

- The username for the local user must not be prefixed with "C##" or "c##".
- The username must be unique within the PDB.
- You can either specify the CONTAINER=CURRENT clause, or omit it, as this is the default setting when the current container is a PDB.

- CONN / AS SYSDBA
-
- -- Switch container while connected to a common user.
- ALTER SESSION SET CONTAINER = pdb1;
-
- -- Create the local user using the CONTAINER clause.
- CREATE USER test_user3 IDENTIFIED BY password1 CONTAINER=CURRENT;
- GRANT CREATE SESSION TO test_user3 CONTAINER=CURRENT;
-
- -- Connect to a privileged user in the PDB.
- CONN system/password@pdb1
-
- -- Create the local user using the default CONTAINER setting.
- CREATE USER test_user4 IDENTIFIED BY password1;
- GRANT CREATE SESSION TO test_user4;

Granting Roles and Privileges to Common and Local Users

The basic difference between a local and common grant is the value used by the CONTAINER clause.

```
-- Common grants.
CONN / AS SYSDBA

GRANT CREATE SESSION TO c##test_user1 CONTAINER=ALL;
GRANT CREATE SESSION TO c##test_role1 CONTAINER=ALL;
GRANT c##test_role1 TO c##test_user1 CONTAINER=ALL;

-- Local grants.
CONN system/password@pdb1
GRANT CREATE SESSION TO test_user3;
GRANT CREATE SESSION TO test_role1;
GRANT test_role1 TO test_user3;
```

Managing Commonly and Locally Granted Privileges

In a multitenant environment, privileges can be granted commonly for an entire CDB or application container, or granted locally to a specific PDB.

How the Oracle Multitenant Option Affects Privileges

In a multitenant environment, all users, including common users, can exercise their privileges only within the current container.

However, a user connected to the root can perform certain operations that affect other pluggable databases (PDBs). These operations include ALTER PLUGGABLE DATABASE, CREATE USER, CREATE ROLE, and ALTER USER. The common user must possess the commonly granted privileges that enable these operations. A common user connected to the root can see metadata pertaining to PDBs by way of the container data objects (for example, multitenant container database (CDB) views and V\$ views) in the root, provided that the common user has been granted privileges required to access these views and his CONTAINER_DATA attribute has been set to allow seeing data about various PDBs. The common user cannot query tables or views in a PDB.

Common users cannot exercise their privileges across other PDBs. They must first switch to the PDB that they want, and then exercise their privileges from there. To switch to a different container, the common user must have the SET CONTAINER privilege. The SET CONTAINER privilege must be granted either commonly or in the container to which the user is attempting to switch.

Be aware that commonly granted privileges may interfere with the security configured for individual PDBs. For example, suppose an application PDB database administrator wants to prevent any user in the PDB from modifying a particular application common object. A privilege (such as UPDATE) granted commonly to PUBLIC or to a common user or common role on the object would circumvent the PDB database administrator's intent.

About Commonly and Locally Granted Privileges

In a multitenant environment, both common users and local users can grant privileges to one another.

Privileges by themselves are neither common nor local. How the privileges are applied depends on whether the privilege is granted commonly or granted locally.

For commonly granted privileges:

- A privilege that is granted commonly can be used in every existing and future container.
- Only common users can grant privileges commonly, and only if the grantee is common.
- A common user can grant privileges to another common user or to a common role.
- The grantor must be connected to the root and must specify CONTAINER=ALL in the GRANT statement.

- Both system and object privileges can be commonly granted. (Object privileges become actual only with regard to the specified object.)
- When a common user connects to or switches to a given container, this user's ability to perform various activities (such as creating a table) is controlled by privileges granted commonly as well as privileges granted locally in the given container.
- Do not grant privileges to PUBLIC commonly.

- GRANT CREATE ANY TABLE TO c##hr_admin CONTAINER=ALL;
- GRANT SELECT ON DBA_OBJECTS TO c##hr_admin CONTAINER=ALL;

For locally granted privileges:

- A privilege granted locally can be used only in the container in which it was granted. When the privilege is granted in the root, it applies only to the root.
- Both common users and local users can grant privileges locally.
- A common user and a local user can grant privileges to other common or local roles.
- The grantor must be connected to the container and must specify CONTAINER=CURRENT in the GRANT statement.
- Any user can grant a privilege locally to any other user or role (both common and local) or to the PUBLIC role.